

IMPROVING PERFORMANCE IN APACHE KAFKA

Kafka performance—Key factors



Four factors affecting Kafka performance

Kafka Burrow
partition lag

Kafka consumer lag

Kafka monitoring

Kafka latency

Kafka optimization

Kafka performance
tuning

Kafka log compaction

Kafka rebalancing

Kafka logs

Kafka lag

Kafka monitoring
tools

Four factors affecting Kafka performance

Apache Kafka® comes with many configurations that developers tweak to achieve maximum performance. While the default values of these parameters are sufficient to handle typical use cases, achieving near real-time performance when dealing with millions of incoming messages requires careful tuning.

This article discusses four factors that affect the performance of Kafka: producer, consumer, broker, and message configurations. We will also cover the settings you should watch out for. Let's get started.

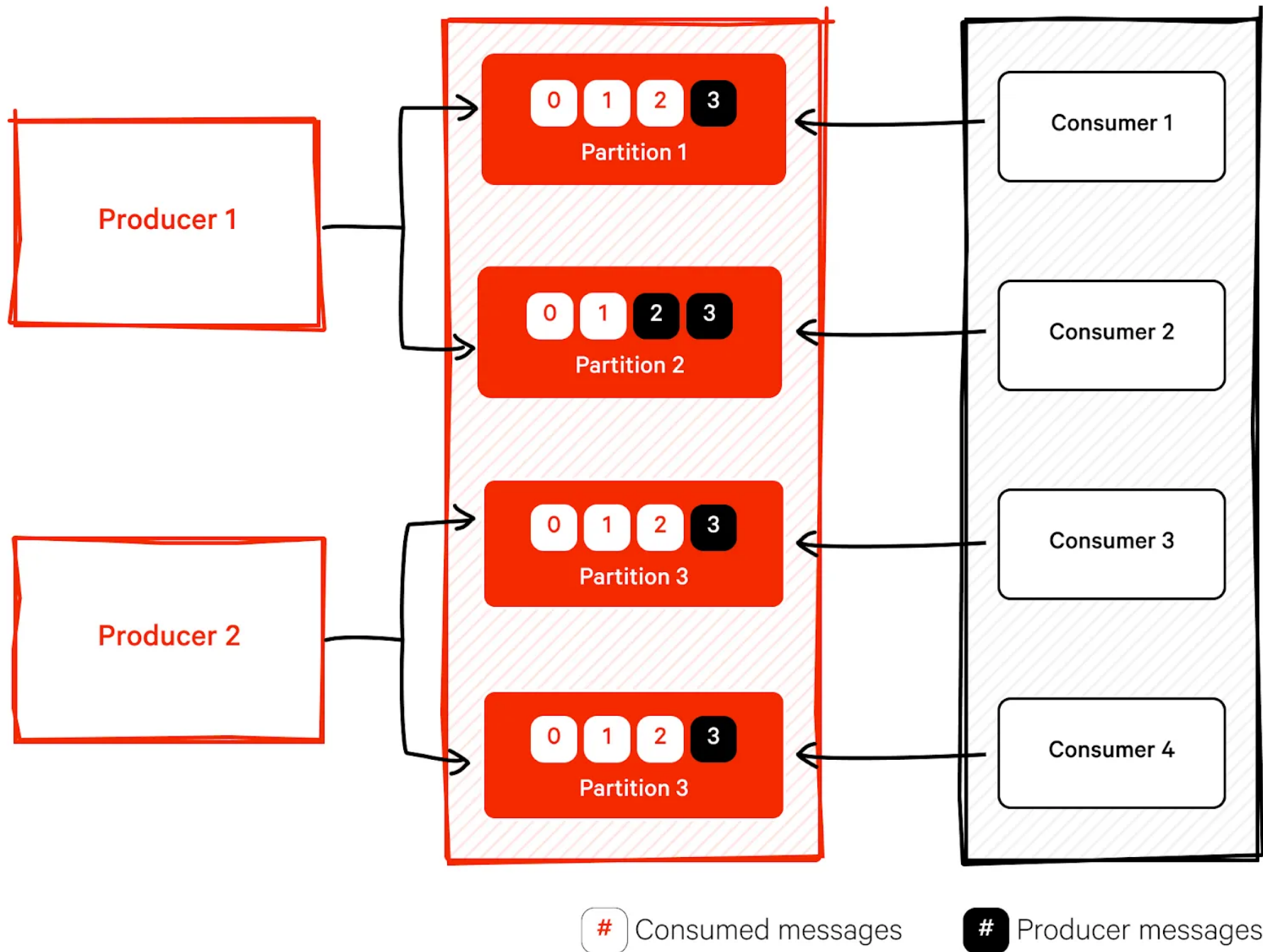
Overview of Kafka basics

This section provides a brief overview of basic Kafka concepts for readers new to Kafka. You can skip to the next section if you are an experienced Kafka user.

Kafka's working model comprises message producers, brokers, and message consumers.

- Producers are client applications that generate data and publish messages to Kafka.
- Brokers store messages durably and enable real-time processing.
- Consumers are client applications that act on the messages and process them.

You can group consumers in Kafka to form consumer groups. Producers and consumers work independently of each other and are fully decoupled.



Apache Kafka working model

Topics and partition

Kafka logically groups messages into topics based on use cases. It stores messages into a partition(s)

inside a single topic. Kafka partitions are thus subsets of data belonging to a topic. They provide concurrency to scale consumers and producers. Kafka ensures that the broker does not deliver already consumed messages to other consumers within the same group. Consumers within a consumer group read messages from different partitions.

Replication

Kafka replicates data across multiple servers to ensure high availability. It keeps multiple copies of each partition in different locations. It defines one of the copies as the leader and other replicas synchronize data from the leader. In addition, Kafka ensures that different brokers store replica partitions. Thus, Kafka replicates data at both the partition level and broker level.

Key Kafka performance metrics

You can measure Kafka’s performance using two main metrics: latency and throughput. You can also benchmark Kafka’s performance in terms of consumer lag.

Latency

Latency is how long it takes for Kafka consumers to fetch a message once it is published to a topic. It is the difference between the time producer sent a message and the consumer received it.

Throughput

Throughput is the number of messages handled by Kafka per unit of time. Achieving ideal values for both throughput and latency is a holy grail and requires optimization of Kafka configuration parameters.

Consumer lag

Consumer lag measures the difference in offset value of the last produced message and the last consumed message.

When to choose Redpanda over Apache Kafka

Start streaming data like it’s 2024.

[LEARN MORE](#)

Summary of key factors affecting Kafka performance

Achieving maximum performance out of Kafka is often a balancing act between durability, availability, throughput, and latency. You can prioritize between these metrics by playing around with the following factors.

Producer configurations	Producer acknowledgment and batching configurations balance throughput and latency.
Consumer configurations	Optimize the number of consumers per group and the amount of data fetched in each pull to improve Kafka performance.
Broker configurations	Kafka partitions and replica configurations balance durability and throughput.
Message configurations	Careful control of message size and message count optimizes Kafka performance.

Factors affecting Kafka performance metrics

1. Producer configurations

Kafka producers write messages in batches according to the defined partition strategy. Kafka replicates messages to multiple brokers according to the replication factor configuration. Once messages are successfully written Kafka broker provides an acknowledgment to producers. You can control all of the above producer behaviors to improve Kafka performance.

Partition strategy

Kafka producers employ different strategies to choose the partitions while writing data. For example:

- Round-robin strategy evenly distributes data across partitions.
- Range partition assignment strategy colocates matching partitions from different topics.
- Sticky partition assignment strategy minimizes partition movements during rebalancing operations.

You should choose a partitioning strategy that ensures evenly distributed partitions, easier

processing, and minimum rebalance movements for your use case. For example, use the range assignment strategy if your use case involves joining topics. On the other hand, if you are working in an environment where adding and reducing consumers are common, choose the sticky partition assignment strategy. Developers can also write custom partition strategies if they want close control of how Kafka writes your data to partitions.

Message batching

Producers write messages in batches to partitions. Batching involves buffering messages till a predefined time or a specific number of messages arrive. This is done to get maximum throughput but increases latency. You can balance the two metrics for your use case by controlling the following configuration parameters.

- **‘batch.size’** controls the number of messages that Kafka producers buffer before writing to a partition
- **‘linger.ms’** defines the time it waits for the batch to get filled.

Broker acknowledgment

The configuration parameter **‘acks’** controls the number of acknowledgments that are required to confirm a successfully written message. It plays an essential role in the throughput you achieve. For example, if message acknowledgments are unnecessary, the producer’s job is quicker but at the cost of message durability.

Redpanda: a powerful Kafka alternative

Fully Kafka API compatible. 6x faster. 100% easier to use.

LEARN MORE

2. Consumer configurations

Kafka consumers fetch messages from the broker and process them. Your partition settings and consumer message batching impact Kafka performance.

Partition settings

Consumers within a group fetch messages from different partitions within a Kafka topic. Kafka maps every partition inside the topic to a single consumer inside the consumer group. However, one consumer inside the consumer group consumes data from multiple partitions. This means in a scenario where there are two partitions and four consumers, two consumers will be idle. In contrast, if your topic has four partitions and two consumers, each consumer has to handle two partitions. The partition-consumer mapping affects the throughput and latency you achieve.

Message batching

Consumers also fetch messages from partitions in batches. The number of messages in a batch and the time the consumer waits to complete a batch affect the throughput and latency. The parameters:

- **‘fetch.min.bytes’** and **‘fetch.max.bytes’** control the minimum and maximum amount of data the Kafka consumer fetches in a single fetch.
- **‘fetch.max.wait.ms’** defines the amount of time the consumer waits to fulfill the **‘fetch.min.bytes’** value

3. Broker configurations

Brokers manage partitions and data replication for scaling. Both configurations impact Kafka performance.

Partitions

A higher number of partitions generally improves the throughput. That said, partitions increase overhead in brokers, and if the broker hardware specifications are not enough to handle the load, both throughput and latency suffer.

Increasing the number of partitions is a complex decision since it affects the message ordering. Kafka guarantees message ordering only to messages coming from the same partition. So if you need to increase the number of partitions, you must define a relevant partition mapping strategy for messages.

Replication

Kafka replicates the data in terms of partitions to ensure durability. Replication is an expensive process, and the configurations that define the nature of replication and the number of replicated copies affect throughput and latency. The configuration parameters

- **‘replication.factor’** controls the number of replicas the broker keeps.
- **‘min.insync.replicas’** defines the number of replicas the producer should save before confirming a message as successfully sent.

Increasing min.insync.replicas degrades both throughput and latency but with the advantage of higher durability. One can opt for asynchronous replication and a minimal number of replicas to get better latency figures. While this ensures better latency performance, it affects data durability since Kafka producers receive acknowledgments earlier than the completion of replication. So the risk of data loss increases.

4. Message configurations

Message size and message count are external factors that affect the performance of a Kafka cluster. A higher number of messages that carry a small amount of data means the broker has to handle a lot more overhead, reducing the total amount of data the broker handles. Therefore, if the use case requires managing a large amount of data in a short time, it is better to organize them as a smaller number of larger messages than as a large number of short messages.

That said, if the success criteria are regarding the number of messages, one has to optimize the broker hardware specifications to handle the overheads. The bottleneck in such cases is the broker CPU.

Have questions about Kafka or streaming data?

Join a global community and chat with the experts on Slack.

JOIN SLACK

Conclusion

How you configure producers, consumers, brokers, and messages affect Kafka’s performance. Kafka partitions help to improve parallel processing, and having the maximum number of partitions that broker hardware can support is always preferable.

Optimizing partition and batching-related parameters of producers and consumers helps one reach acceptable levels of throughput and latency. However, these parameters affect throughput and latency in opposite directions and must be carefully tuned according to the use case. Relaxing configurations related to replication and acknowledgments improve throughput and latency at the cost of availability and durability. You can learn more about fine-tuning in our article on [Kafka performance tuning](#).

Redpanda Serverless: from zero to streaming in 5 seconds

Just sign up, spin up, and start streaming data!

FREE TRIAL

Chapters